Creation of Azure Cosmos DB Account and Container

Objective

The objective of this exercise is to understand how to create an Azure Cosmos DB account, create a database and container inside it, and add items using Azure Portal. This exercise also helps in understanding basic concepts of NoSQL database in cloud environment.

Introduction

Azure Cosmos DB is a globally distributed NoSQL database service provided by Microsoft Azure. Unlike traditional SQL databases, Cosmos DB does not require a fixed schema and allows flexible data storage in JSON format.

In this exercise, I learned how to create a Cosmos DB account, configure basic settings, and work with Data Explorer to insert and query documents.

Step 1 – Creating Azure Cosmos DB Account

First, we open the Azure Portal and click on **Create a Resource**.
Then we search for **Azure Cosmos DB** and select the option for **Azure Cosmos DB for NoSQL**.

While creating the account, the following details are entered:

- Subscription – Selected active subscription
- Resource Group – Created new or selected existing resource group
- Account Name – Given a unique name (only lowercase letters and numbers allowed)
- Location – Selected nearest region
- Capacity Mode – Selected Provisioned Throughput
- Free Tier – Enabled if available

Other settings like Geo-redundancy, Multi-region writes, and Availability Zones are kept as default (Disabled).

After reviewing all details, we click on **Create**.
It takes a few minutes for deployment to complete.

Step 2 – Creating Database and Container

After the account is created, we open the resource and select **Data Explorer**.

Inside Data Explorer:

1. Click on **New Container**.
2. Enter Database ID as **ToDoList**.
3. Select **Manual throughput**.
4. Set throughput to **400 RU/s**.
5. Enter Container ID as **Items**.

6. Set Partition Key as **/category**.

Then click **OK**.

Now the database and container are successfully created.

At this stage, I understood that the partition key is important for distributing data properly in Cosmos DB.

Step 3 – Adding Items (Documents)

Inside the **Items** container, we click on **New Item**.

We enter the following JSON document:

```
{
   "id": "1",
   "category": "personal",
   "name": "groceries",
   "description": "Pick up apples and strawberries.",
   "isComplete": false
}
```

After entering the document, we click **Save**.

Then another item is created with a different id and some different values.

I observed that Cosmos DB allows flexible document structure, which means we do not need to follow a strict schema like SQL database.

Step 4 – Running Queries

In Data Explorer, the default query:

```
SELECT * FROM c
```

is used to display all documents in the container.

Then the query is modified to:

```
ORDER BY c._ts DESC
```

This query shows the latest inserted documents first.

By running these queries, I understood how Cosmos DB retrieves and sorts data.

Conclusion

From this exercise, I learned how Azure Cosmos DB works as a NoSQL cloud database service. I understood the process of creating an account, database, and container. I also learned how partition keys help in organizing data and how JSON documents are stored inside containers.

Overall, this exercise gave me basic understanding of working with NoSQL databases in Azure portal.